

Ravan Muradzada

- 1) An **IP address** in your Linux machine is the **network identity** of that machine.

The command is `ip a`. The result is:

```
[root@mhd1 b]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:2e:ea:ea brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    inet 192.168.88.129/24 brd 192.168.88.255 scope global dynamic noprefixroute ens160
        valid_lft 1734sec preferred_lft 1734sec
    inet6 fe80::20c:29ff:fe2e:ea:ea/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

From here, we take `192.168.88.129` as ip.

- 2) We wrote `ssh username@ip_address`. In our example, we need to write:

`ssh root@192.168.88.129`.

- 3) The command will be like that:

`netstat -tuln | grep LISTEN`

In our virtual machines, there is no netstat command downloaded, so I cannot show its output here. In this command, the first part gets open ports, grep find lines with `LISTEN` from the list.

- 4) First, I need to see the open ports in linux machine. To see it, we need to type this command: `ss -tuln | cat`. There are both udp and tcp ports, however with telnet we cannot reach udp ports. From this command, I found one open port (with tcp), it is 22.

```
[root@mhd1 ~]# ss -tuln | cat
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
udp UNCONN 0 0 127.0.0.1:323 0.0.0.0:*
udp UNCONN 0 0 [::1]:323 [::]:*
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
tcp LISTEN 0 128 [::]:22 [::]:*
[root@mhd1 ~]#
```

We also need to have ip address of linux machine. So we write `'ip a'` command again and take ip. (We learned it detailed in Question 1).

To try to connect from windows machine, I opened cmd and type

`'telnet <IP> <PORT>':`

```
C:\Users\ravan> telnet 192.168.88.129 323|
```

It is successful and the result:

```
SSH-2.0-OpenSSH_8.7
```

5) To check which directory I am, we need to type `'pwd'` command which means `'print working directory'`. It locates in `'/bin'` folder.

6) I tried, but because of subscription problems, I failed to install it.

7) We need to use `'mkdir -p':`

```
[root@mhd1 ~]# mkdir -p /folder1/f2/f3/f4/f5
```

8) It tries to remove `'/'` (whole system) without asking, because we use `'-f'` which is forcing to delete everything in the system. However, in modern systems that command directly does not work, we need to add another option like `'--no-preserve-root'` to really delete whole system.

```
[root@mhd1 f5]# cd /tmp
[root@mhd1 tmp]# cd /etc/sysconfig/network-scripts
[root@mhd1 network-scripts]# ls
readme-ifcfg-rh.txt
```

9)

- a. `cd -` travels between the last two directories we are.

```
[root@mhd1 a]# cd ~/b
[root@mhd1 b]# cd -
/root/a
[root@mhd1 a]# cd -
/root/b
[root@mhd1 b]# cd -
/root/a
[root@mhd1 a]# cd -
/root/b
[root@mhd1 b]#
```

Here initially we were in `a` and then went to `b` directory. Now these are the last two directories we travelled. So when we run `cd -`, it will go `a` and `b` sequentially.

`cd ..`

basically goes the parent directory.

```
[root@mhd1 B]# pwd
/root/b/B
[root@mhd1 B]# cd ..
[root@mhd1 b]# pwd
/root/b
```

- b. `cd ~` goes to home directory of the current user. `cd .` just stays the same directory, because `..` means the current directory. To check which directory I am, I write `pwd`

```
[root@mhd1 child]# pwd
/root/b/parent/child
[root@mhd1 child]# cd ..
[root@mhd1 parent]# pwd
/root/b/parent
[root@mhd1 parent]# cd .
[root@mhd1 parent]# pwd
/root/b/parent
```

-
- 10) `rm` itself removes files from the system. If we want to remove directories, we need to use `-r` which means `recursive`. When we try to remove files or directories from the system it asks whether we are sure to remove it or not. If we

really to continue and remove, we press `y` to remove. Usage of `-f` option with `rm` prevents this question, so the command removes it without asking.

11)

```
[root@mhd1 ~]# touch rhcsa.red
[root@mhd1 ~]# mv rhcsa.red rhcsa.blue
[root@mhd1 ~]# ll rhcsa.blue
-rw-r--r--. 1 root root 0 Feb 24 14:27 rhcsa.blue
[root@mhd1 ~]# cp -p rhcsa.blue /tmp/
[root@mhd1 ~]# ll /tmp/rhcsa.blue
-rw-r--r--. 1 root root 0 Feb 24 14:27 /tmp/rhcsa.blue
```

The option we need to use is `-p` which means 'preserve'.

12) `df` command shows us how much disk space is used and free on our system.

We can use `-h` option to make the result more human-readable.

`du` command checks the size of the directory or file, not the whole disk. If we use it like `du -sh`, it shows the size of the current directory. However, if we use it like `du -sh *`, then it shows the size of each file in the current directory.

```
[root@mhd1 ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M   0    4.0M   0% /dev
tmpfs           838M   0    838M   0% /dev/shm
tmpfs           335M   5.0M  330M   2% /run
efivarfs        256K   57K   195K  23% /sys/firmware/efi/efivars
/dev/mapper/rhel-root 8.4G  1.7G  6.8G  20% /
/dev/nvme0n1p2   960M  244M  717M  26% /boot
/dev/nvme0n1p1   599M   7.4M  592M   2% /boot/efi
tmpfs           168M   0    168M   0% /run/user/0
[root@mhd1 ~]# du -sh
40K .
[root@mhd1 ~]# du -sh *
0 a
4.0K anaconda-ks.cfg
0 b
4.0K dir1
4.0K file.txt
0 rhcsa.blue
```

13) a. Red Hat 9.7; b. `#1 SMP PREEMPT_DYNAMIC Fri Oct 17 14:16:35 EDT 2025`; c. We can use `uname -v` to see kernel version, also we can use `uname -a` to see more information about the system.

14) We can use whether `exit` or `logout` command. It is possible to press `Ctrl+D` too.

15)

```
[root@mhd1 ~]# find /etc -name sshd_config  
/etc/ssh/sshd_config
```

16)

```
[root@mhd1 ~]# find /etc -name sshd_config -exec head -n15 {} \; | tail -n1  
Include /etc/ssh/sshd_config.d/*.conf
```

It first finds the file itself from the system, then sends the output to `head` with exec feature of `find` command, then output is sent to `tail` as input.

If we already know where the file is located, we can just write like that:

```
[root@mhd1 ~]# head -n15 /etc/ssh/sshd_config | tail -n1  
Include /etc/ssh/sshd_config.d/*.conf
```

```
[root@mhd1 ~]# cp /etc/hosts /root/mydir/
[root@mhd1 ~]# ll mydir/
total 4
-rw-r--r--. 1 root root 158 Feb 24 14:50 hosts
[root@mhd1 ~]# tar -cf arch.tar mydir/
[root@mhd1 ~]# ls
a  anaconda-ks.cfg  arch.tar  b  dir1  file.txt  mydir
[root@mhd1 ~]# tar -tf arch.tar
mydir/
mydir/hosts
```

17)

18) When I run history command, it shows my command history. When I run `!!20`, it will run 20th command in history.

- `!!` runs the last command again.
- It looks at my history, filters and shows all usages of `grep` in history:

```
[root@mhd1 ~]# history | grep grep
  5  netstat -tuln | grep LISTEN
 11  ls /bin | grep pwd
 21  ls /sbin/ | grep pwd
154  history | grep grep
```

19)

- ```
[root@mhd1 ~]# grep -r "port" /etc/ssh/
/etc/ssh/ssh_config.d/50-redhat.conf:# to the original X11 display. As virtually no X11 client supports the untrusted
/etc/ssh/ssh_config:# If you want to change the port on a SELinux system, you have to tell
/etc/ssh/sshd_config:# semanage port -a -t ssh_port_t -p tcp #PORTNUMBER
/etc/ssh/sshd_config:# WARNING: 'UsePAM no' is not supported in RHEL and may cause several
```
- ```
[root@mhd1 ~]# grep -ri "port" /etc/ssh/
/etc/ssh/ssh_config.d/50-redhat.conf:# to the original X11 display. As virtually no X11 client supports the untrusted
/etc/ssh/ssh_config:# Port 22
/etc/ssh/sshd_config:# If you want to change the port on a SELinux system, you have to tell
/etc/ssh/sshd_config:# semanage port -a -t ssh_port_t -p tcp #PORTNUMBER
/etc/ssh/sshd_config:#Port 22
/etc/ssh/sshd_config:# WARNING: 'UsePAM no' is not supported in RHEL and may cause several
/etc/ssh/sshd_config:#GatewayPorts no
```

- c. ``-A n`` shows n lines after the match. ``-B`` shows n lines before the match. ``-C`` shows n lines before and after the match. ``-o`` only print the matching part. ``-w`` match whole words only.
-

20)

```
[root@mhd1 ~]# w
 15:33:02 up  1:24,  2 users,  load average: 0.00, 0.00, 0.00
USER      TTY      LOGIN@  IDLE   JCPU   PCPU   WHAT
root      tty1     13:46   1:46m  0.00s  0.00s  -bash
root      pts/0    14:40   3.00s  0.06s  0.00s  w
[root@mhd1 ~]# who
root      tty1           2026-02-24 13:46
root      pts/0          2026-02-24 14:40 (192.168.88.1)
[root@mhd1 ~]# whoami
root
[root@mhd1 ~]# who am i
root      pts/0          2026-02-24 14:40 (192.168.88.1)

A.
[root@mhd1 ~]# tail --follow /var/log/secure
Feb 24 16:39:28 mhd1 su[2320]: pam_unix(su-l:session): session opened for user root(uid=0) by root(uid=0)
Feb 24 16:39:33 mhd1 su[2351]: pam_unix(su-l:session): session opened for user root(uid=0) by root(uid=0)
Feb 24 16:39:37 mhd1 su[2380]: pam_unix(su-l:session): session opened for user root(uid=0) by root(uid=0)
Feb 24 16:40:24 mhd1 su[2412]: pam_unix(su-l:session): session opened for user root(uid=0) by root(uid=0)
Feb 24 16:41:42 mhd1 su[2467]: pam_unix(su-l:session): session opened for user root(uid=0) by root(uid=0)
Feb 24 16:42:10 mhd1 sshd[2498]: Accepted password for root from 192.168.88.1 port 57944 ssh2
Feb 24 16:42:10 mhd1 sshd[2498]: pam_unix(sshd:session): session opened for user root(uid=0) by root(uid=0)
Feb 24 16:42:11 mhd1 sshd[2502]: Accepted password for root from 192.168.88.1 port 57945 ssh2
Feb 24 16:42:11 mhd1 sshd[2502]: pam_unix(sshd:session): session opened for user root(uid=0) by root(uid=0)
Feb 24 16:42:21 mhd1 su[2561]: pam_unix(su-l:session): session opened for user root(uid=0) by root(uid=0)
Feb 24 16:43:40 mhd1 su[2596]: pam_unix(su-l:session): session opened for user root(uid=0) by root(uid=0)

B.
```

When I use ``--follow`` option I can track the changes live.

21) It tries to install everything starting with ``bash``.

22) ``history`` basically shows the command history. ``man`` is the command to get detailed information about any command, we type it like that ``man pwd``. ``--help``

is also very useful to learn detailed about commands, however it has less details than ``man`` command.

23)

- a. To see all files including hidden ones, we need to type ``ls -a`` where ``-a`` means all files.
- b. To create a hidden file, we need to name file starting ``.``, like ``.`my_hidden_file.txt``.

```
[root@mhd1 ~]# touch .my_hidden_file.txt
[root@mhd1 ~]# ls
a  anaconda-ks.cfg  arch1.tar  arch.tar  b  dir1  file.txt  mydir  rhcsa.blue
[root@mhd1 ~]# ls -a
.  a  arch1.tar  b  .bash_logout  .bashrc  dir1  .lessht  .my_hidden_file.txt  .ssh
.. anaconda-ks.cfg  arch.tar  .bash_history  .bash_profile  .cshrc  file.txt  mydir  rhcsa.blue  .tcshrc
```

``ls`` itself cannot show hidden files, but ``ls -a`` can.

24) ``>`` redirects and overwrites, so it removes the content and then adds new content. So, after the redirection in bio.txt, there will be "Linux was created by Linus Torvalds".

``>>`` redirects and appends without overwriting. So, after the redirections in session.txt, there will be

"IDTECH RHCSA Session"

"IDTECH RHCE and DevOPS Session"

25) ``a=5`` and ``color=blue`` created variables and assign value to them. ``echo $a`` and ``echo $color`` prints the values of the variables. However, ``echo a`` or ``echo color`` prints just the strings of ``a`` and ``color``.